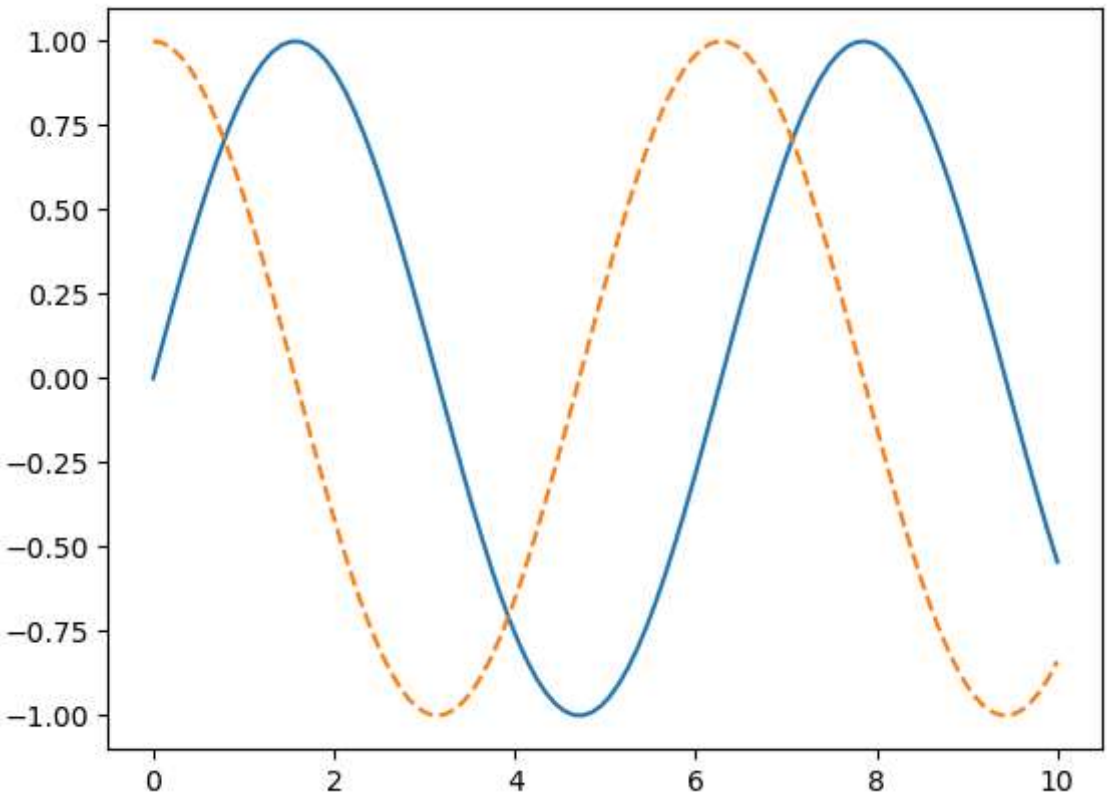
Matplotlib

In [2]:
```
import numpy as np
import pandas as pd
```

In [6]:
```
import matplotlib.pyplot as plt
```

In [12]:
```
# % matplotlib inline

x1 = np.linspace(0, 10, 100)

fig = plt.figure()          # create plot figure


plt.plot(x1, np.sin(x1), '-')
plt.plot(x1, np.cos(x1), '--')
```

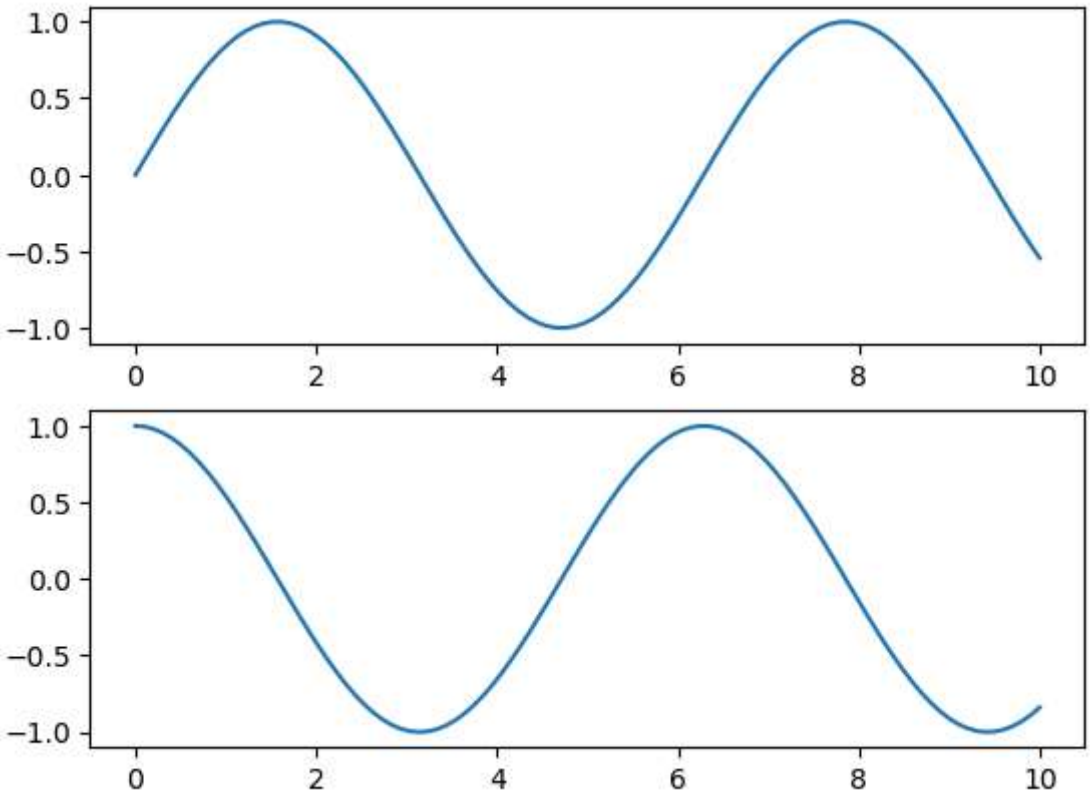
Out[12]:
```
[<matplotlib.lines.Line2D at 0x1525fee5c10>]
```



In [18]:
```
# create a plot figure
plt.figure()

# create the first of two panels and set current axis
plt.subplot(2,1,1) # rows, column, panel number
plt.plot(x1, np.sin(x1))

# create the second of two panels and set current axis
```

```
plt.subplot(2,1,2) # rows, column, panel number
plt.plot(x1, np.cos(x1))
```

Out[18]: [<matplotlib.lines.Line2D at 0x15260901ac0>]

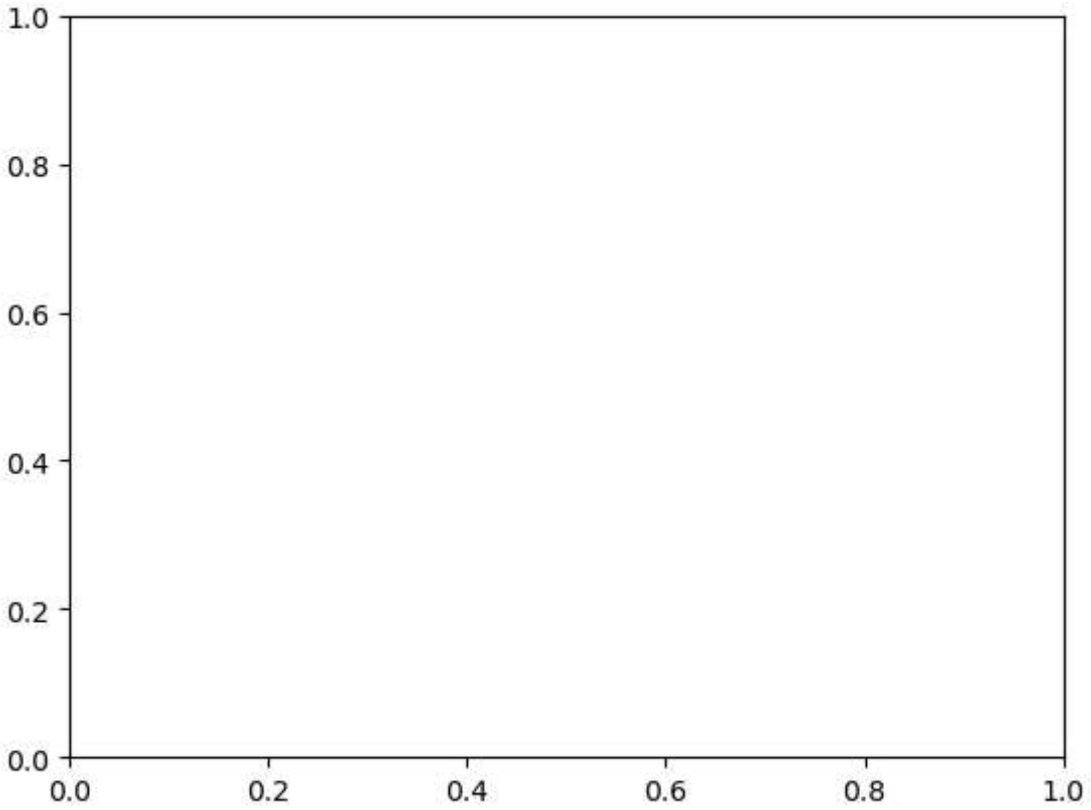


In [30]:
```
# get current figure info

print(plt.gcf())
```

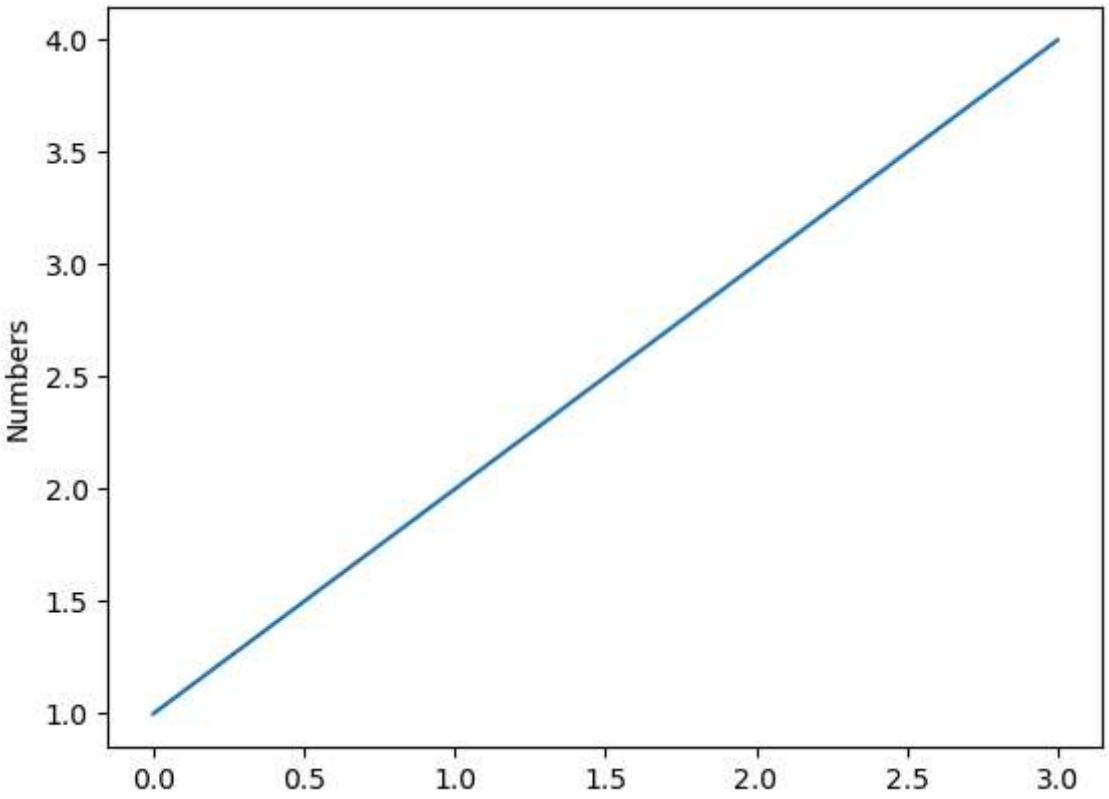
```
Figure(640x480)
<Figure size 640x480 with 0 Axes>
```

In [32]:
```
# get current axis info

print(plt.gca())
```

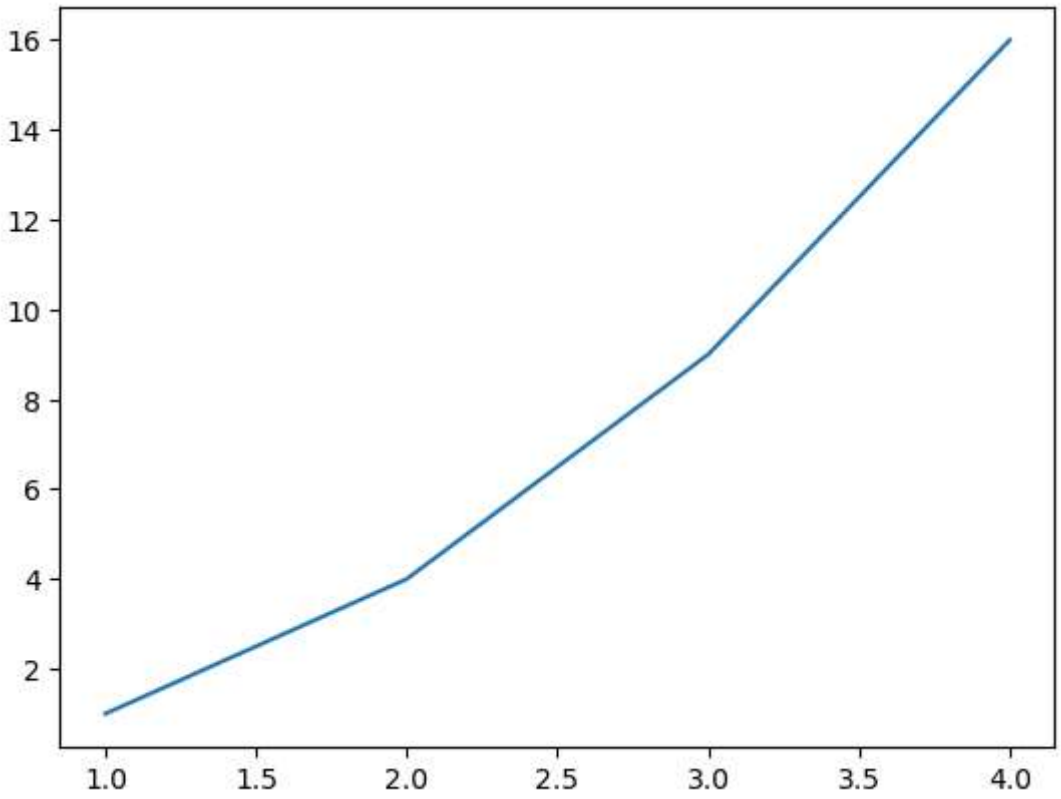
```
Axes(0.125,0.11;0.775x0.77)
```



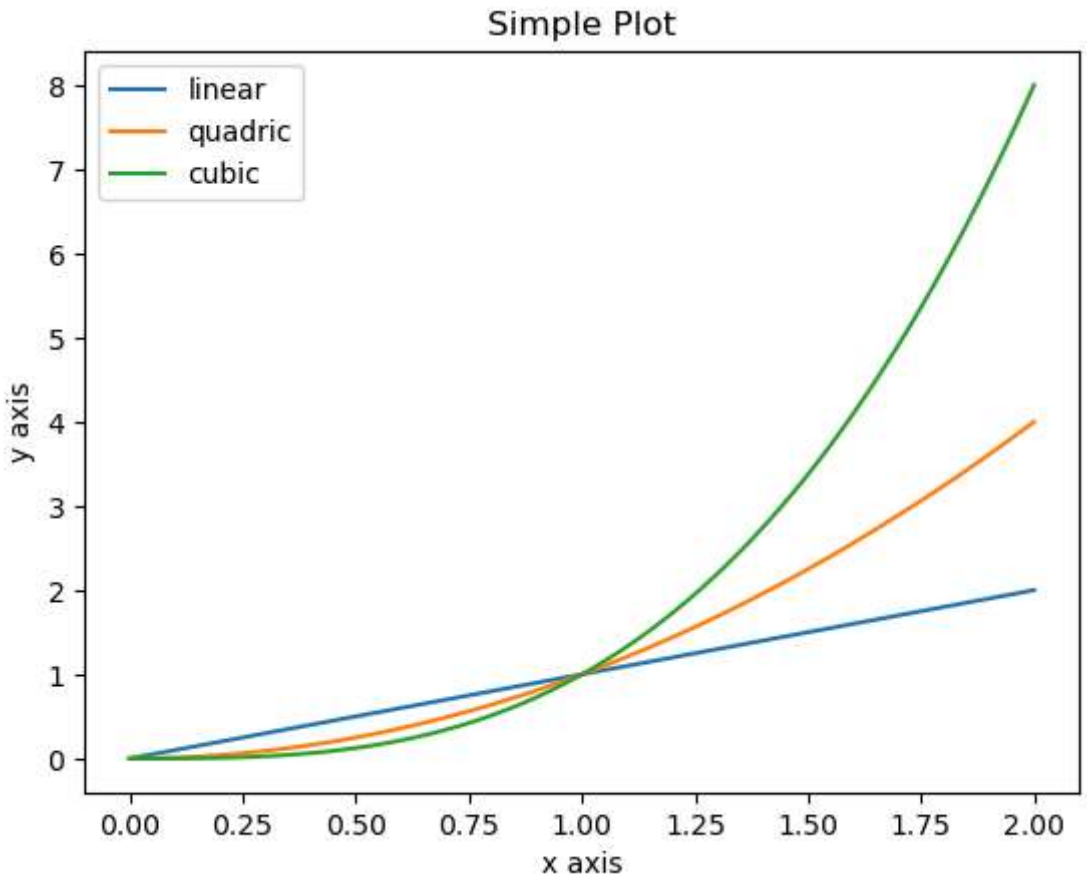
In [34]:
```
plt.plot([1, 2, 3, 4])
plt.ylabel('Numbers')
plt.show()
```



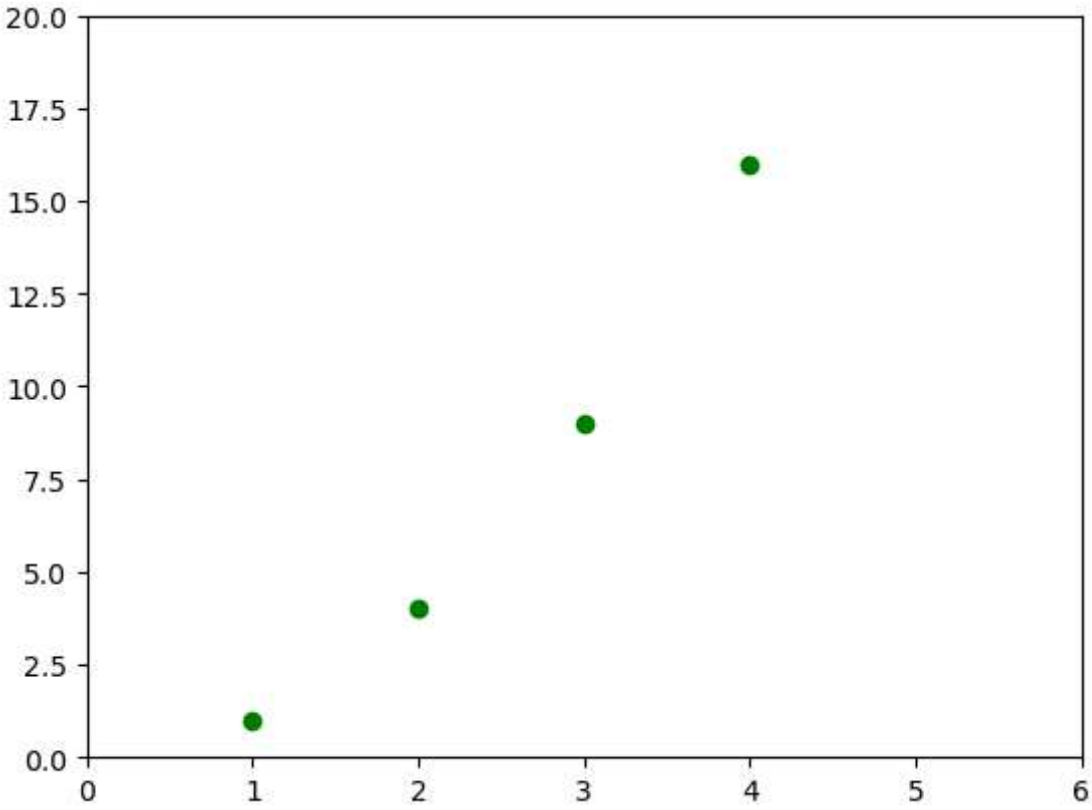
In [36]:
```
plt.plot([1,2,3,4],[1,4,9,16])
plt.show()
```



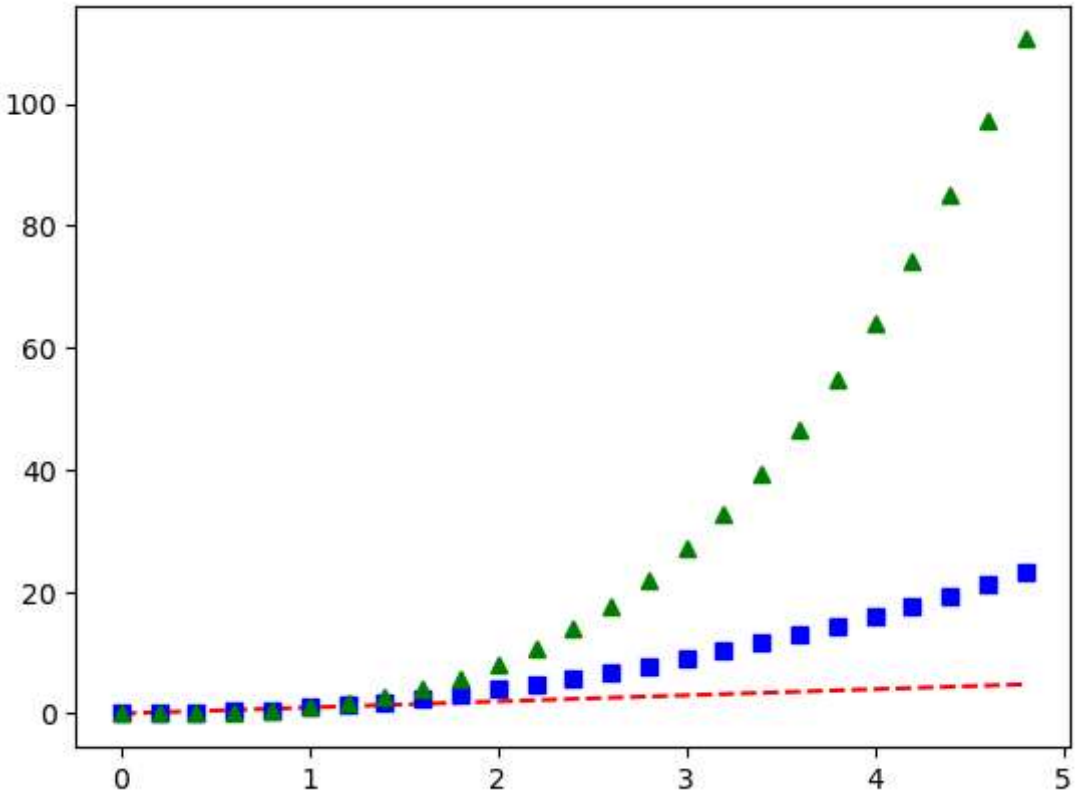
In [40]:
```
x = np.linspace(0,2,100)

plt.plot(x, x, label='linear')
plt.plot(x, x**2, label='quadric')
plt.plot(x, x**3, label='cubic')

plt.xlabel('x axis')
plt.ylabel('y axis')

plt.title('Simple Plot')
plt.legend()
plt.show()
```



In [42]:
```
plt.plot([1,2,3,4] , [1,4,9,16] , 'go')          # green o datapoint
plt.axis([0, 6, 0, 20])                          # controls both x and y axis; [xmin
plt.show()
```

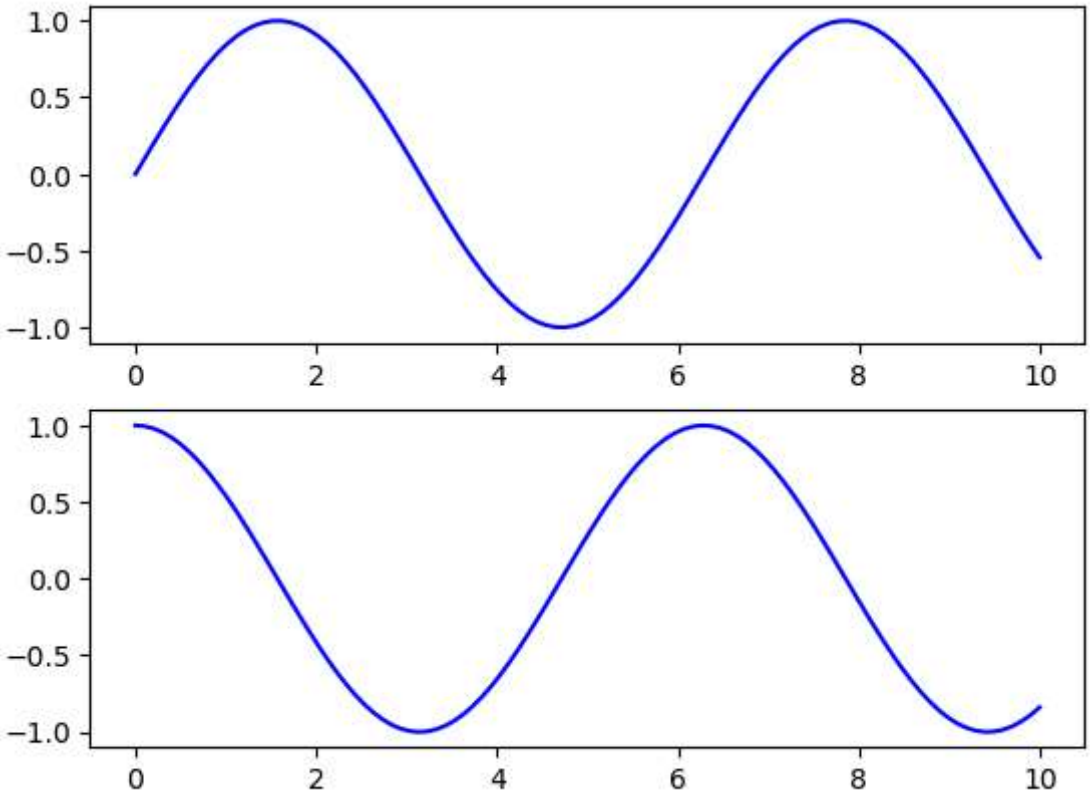


In [44]:
```
# evenly sampled time at 200ms intrvals
t = np.arange(0., 5., 0.2)

# red dashes, blue squares, and green triangles

plt.plot(t,t,'r--', t, t**2, 'bs', t, t**3, 'g^')
plt.show()
```



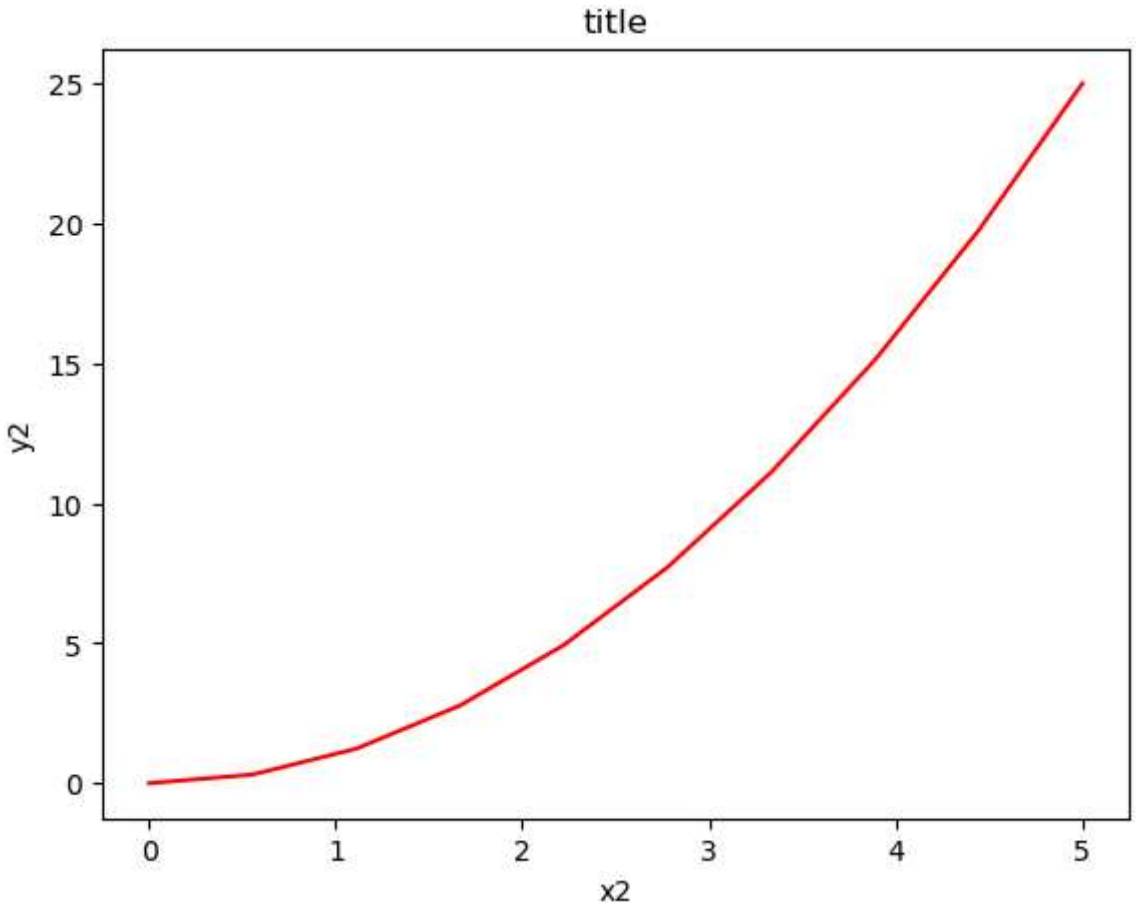
In [46]:
```
# First create grid of plots
# ax will be an array of two Axes objects
fig,ax = plt.subplots(2)            # row-2; col-1 (default if not specified)

# call plot() method on the appropriate object
ax[0].plot(x1,np.sin(x1),'b-')
ax[1].plot(x1,np.cos(x1),'b-')
```

Out[46]:
```
[<matplotlib.lines.Line2D at 0x1525ffb5cd0>]
```

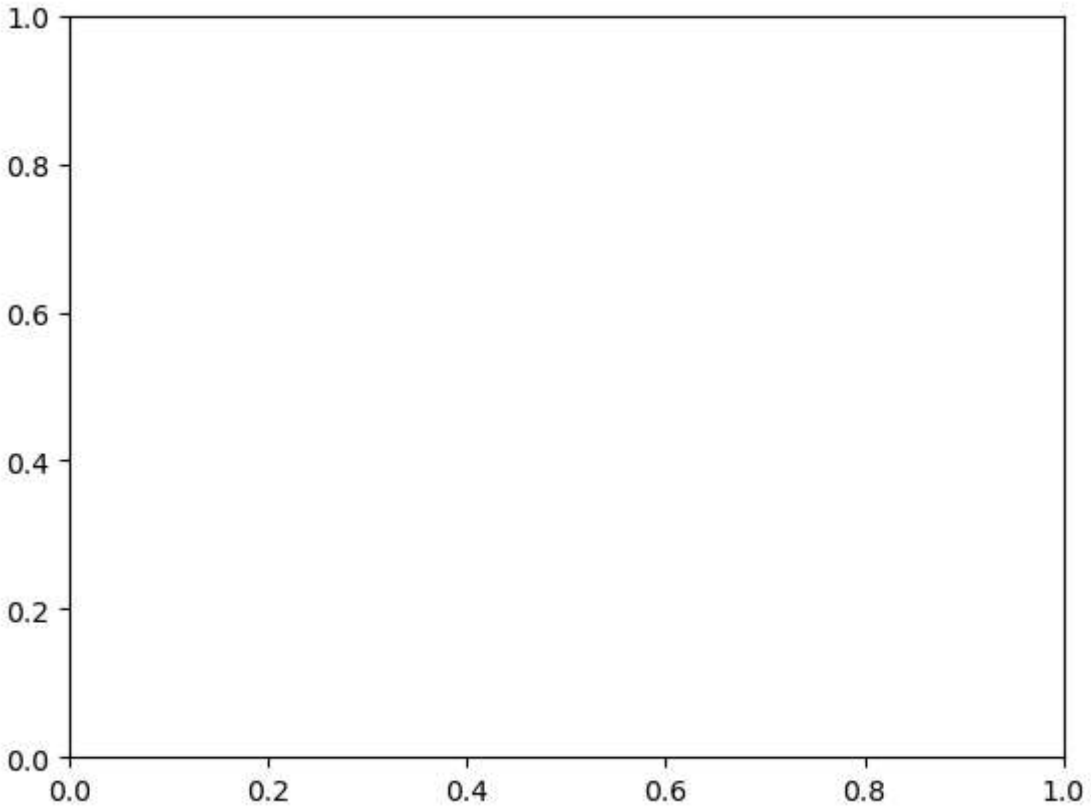


In [48]:
```
fig = plt.figure()

x2 = np.linspace(0, 5, 10)
y2 = x2 ** 2

axes = fig.add_axes([0.1,0.1,0.8,0.8])

axes.plot(x2,y2,'r')              # this approch use to get more control

axes.set_xlabel('x2')
axes.set_ylabel('y2')
axes.set_title('title')
```

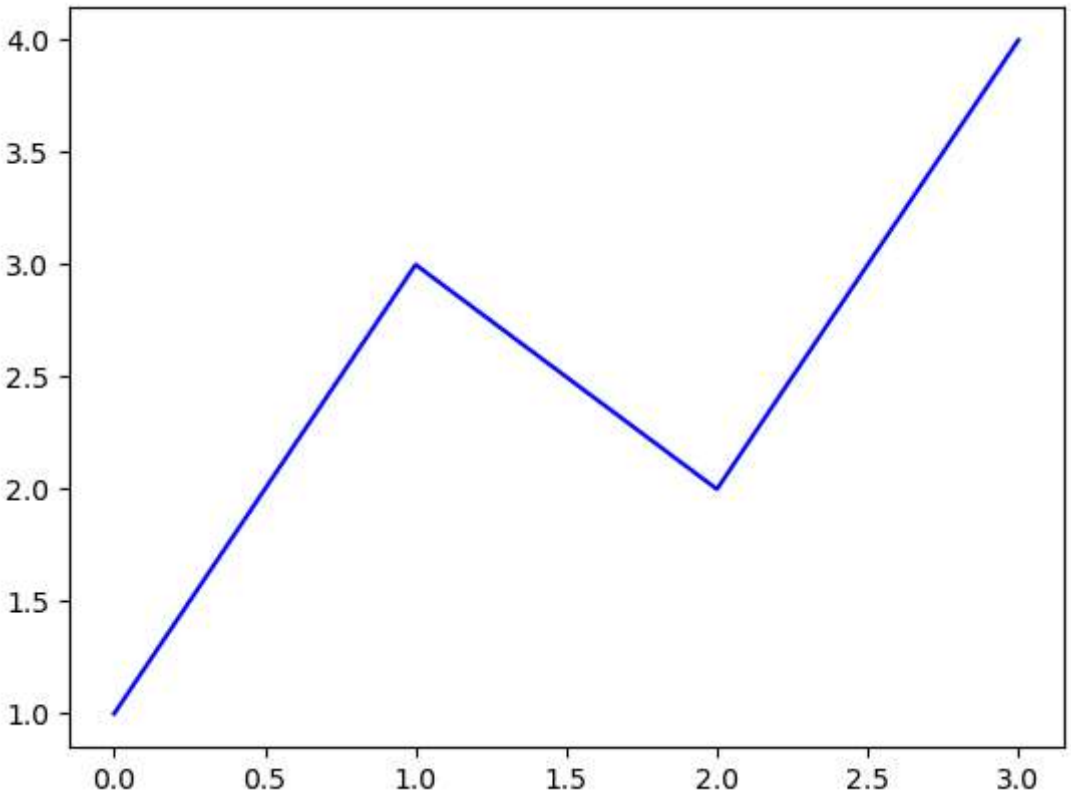
Out[48]:
```
Text(0.5, 1.0, 'title')
```



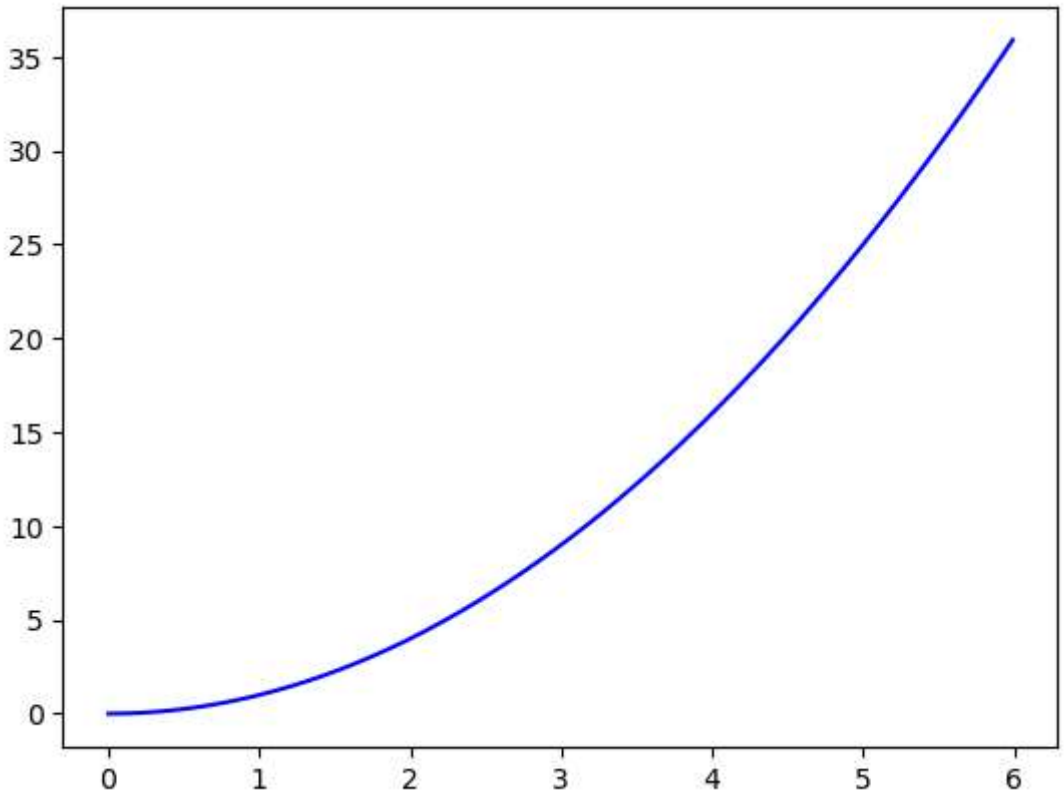
In [50]:
```
fig = plt.figure()

ax = plt.axes()
```



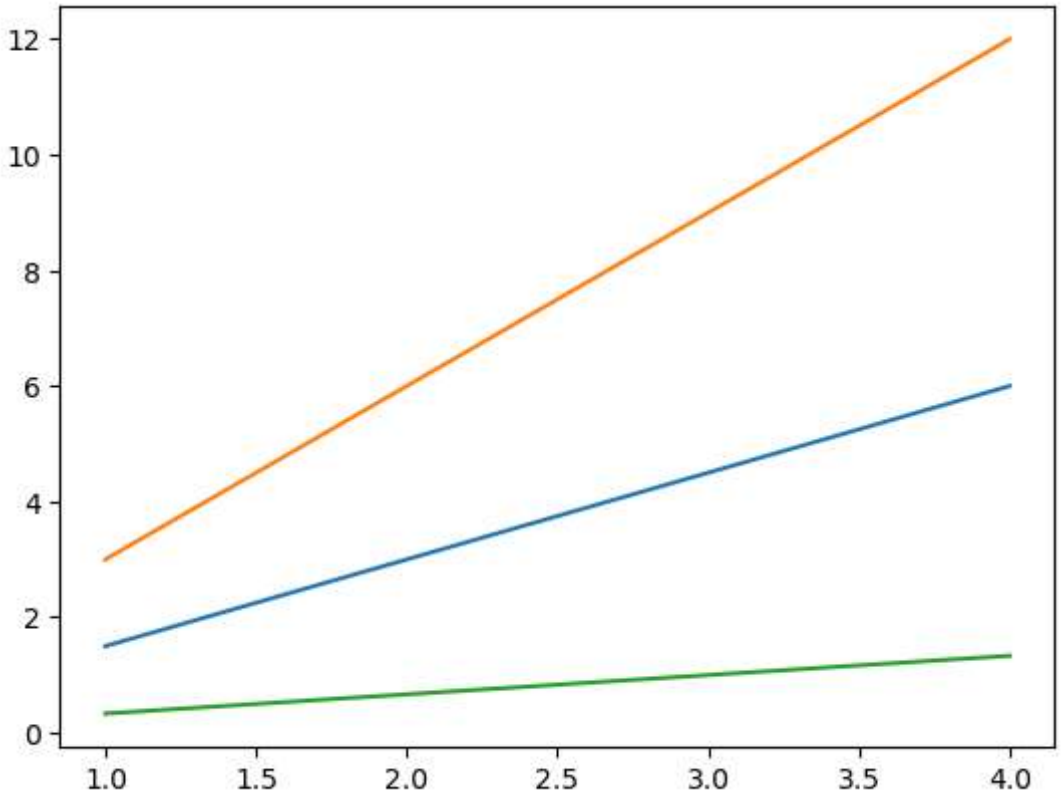
In [52]:
```
plt.plot([1,3,2,4],'b-')

plt.show()
```



In [56]:
```
x3 = np.arange(0.0, 6.0, 0.01)

plt.plot(x3, [xi**2 for xi in x3], 'b-')         # xi**2 for xi in x3 : square of e
plt.show()
```



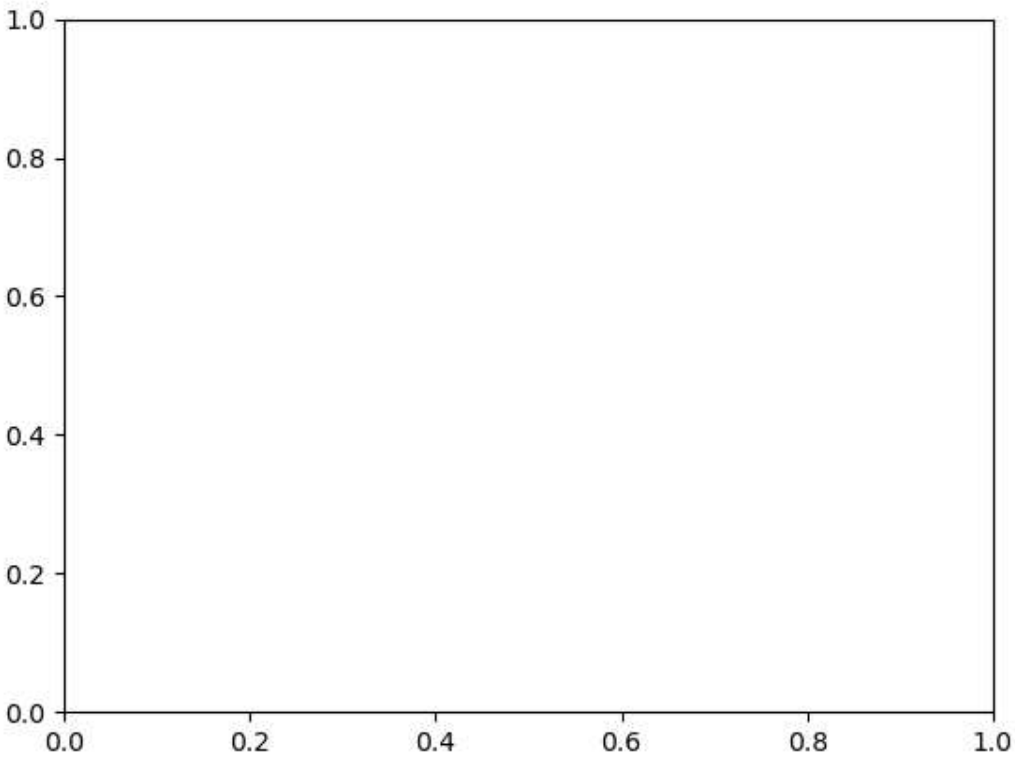
In [58]:
```
x4 = range(1, 5)

plt.plot(x4, [xi*1.5 for xi in x4])

plt.plot(x4, [xi*3 for xi in x4])

plt.plot(x4, [xi/3.0 for xi in x4])

plt.show()
```



In [60]:
```
# saving the figure

fig.savefig('plot1.png')
```

In [62]:
```
# Explore the contents of figure

from IPython.display import Image
Image('plot1.png')
```

Out[62]:



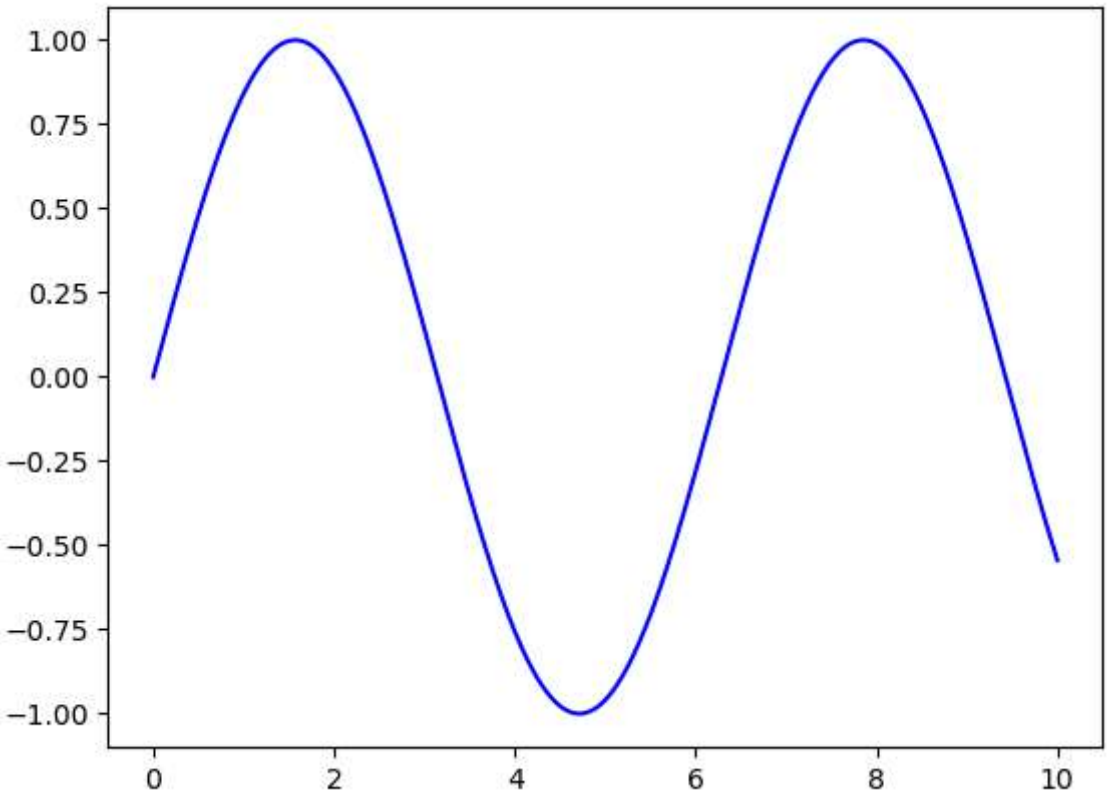
In [64]:
```
# Explore supported file formats

fig.canvas.get_supported_filetypes()
```

Out[64]:
```
{'eps': 'Encapsulated Postscript',
 'jpg': 'Joint Photographic Experts Group',
 'jpeg': 'Joint Photographic Experts Group',
 'pdf': 'Portable Document Format',
 'pgf': 'PGF code for LaTeX',
 'png': 'Portable Network Graphics',
 'ps': 'Postscript',
 'raw': 'Raw RGBA bitmap',
 'rgba': 'Raw RGBA bitmap',
 'svg': 'Scalable Vector Graphics',
 'svgz': 'Scalable Vector Graphics',
 'tif': 'Tagged Image File Format',
 'tiff': 'Tagged Image File Format',
 'webp': 'WebP Image Format'}
```

In [66]:
```
# Create figure and axes first

fig = plt.figure()

ax = plt.axes()          # Create a new set of axes (a plot area) within the curren

# Declare a variable x5
x5 = np.linspace(0,10,1000)
```

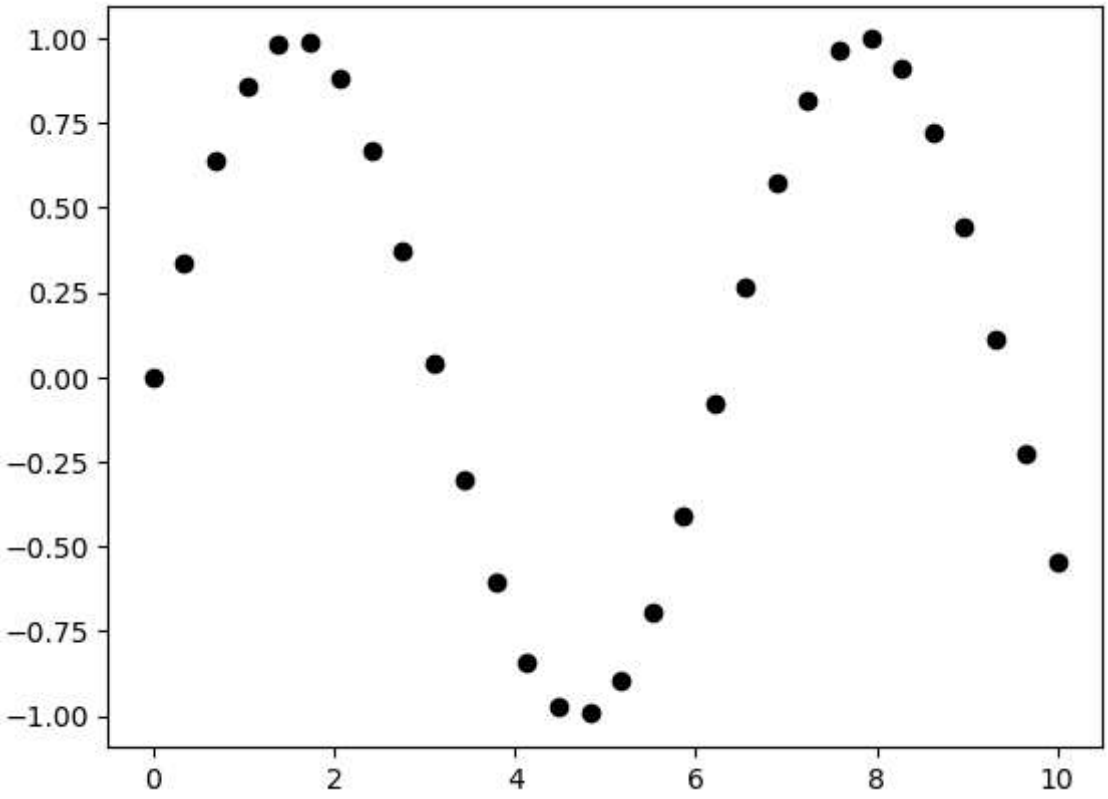
```
# Plot the sinusoid function
ax.plot(x5, np.sin(x5), 'b-')
```

Out[66]: [<matplotlib.lines.Line2D at 0x15261ead8b0>]

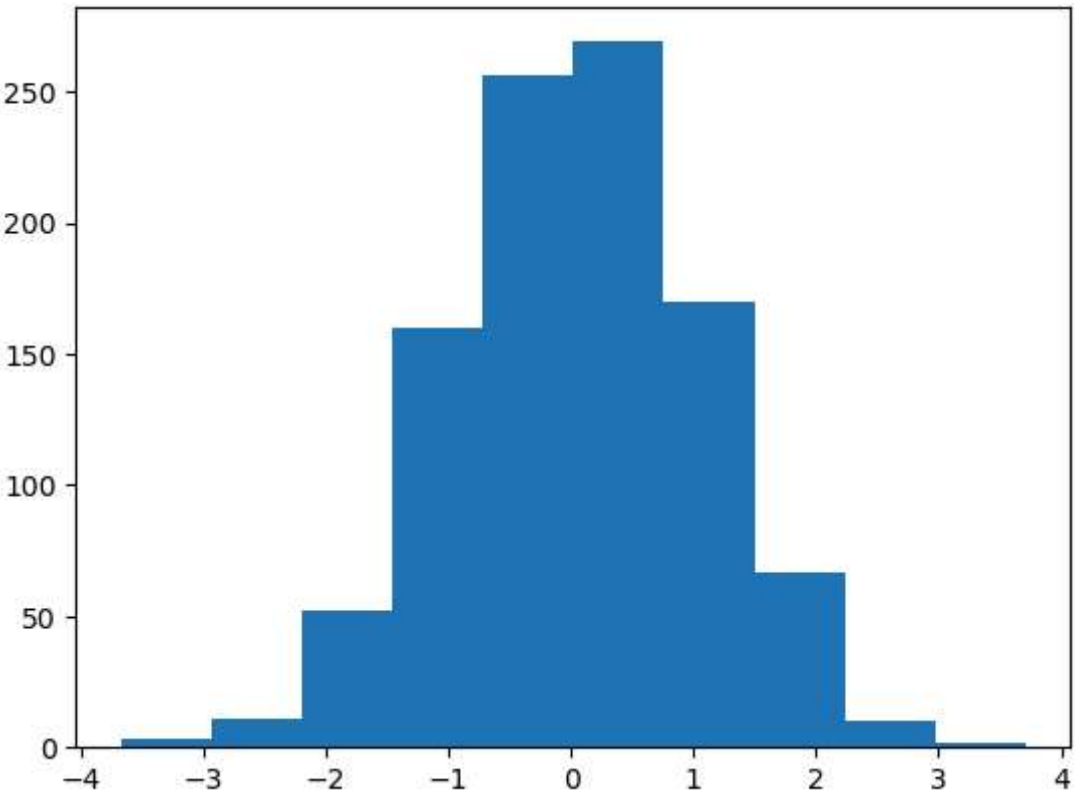


In [70]:
```
x7 = np.linspace(0,10,30)
y7 = np.sin(x7)

plt.plot(x7,y7,'ko')     # ko- black color, 'o' datapoint
```

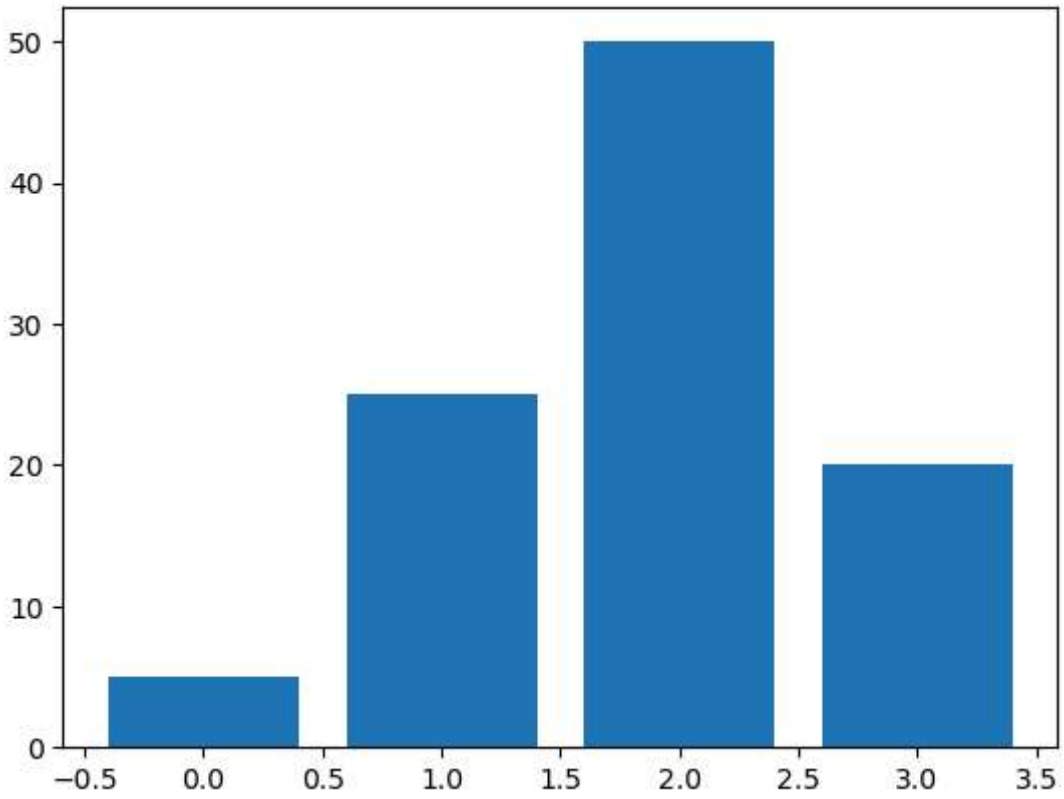
Out[70]: [<matplotlib.lines.Line2D at 0x152620e0a70>]



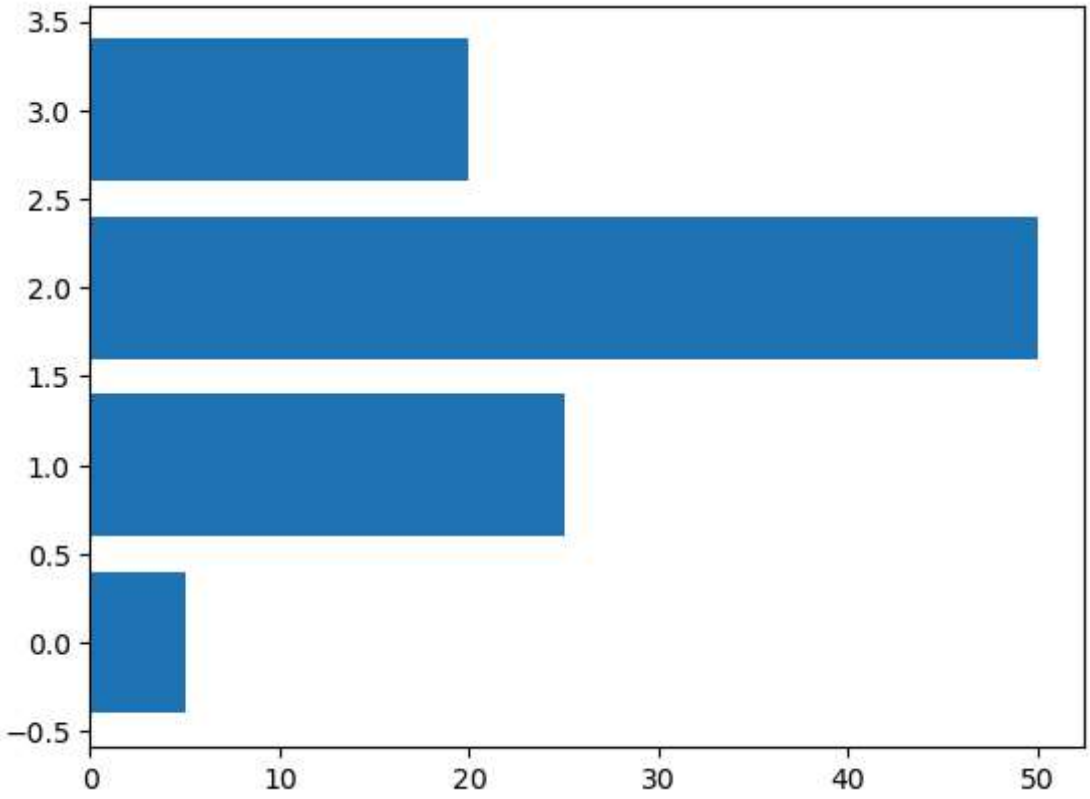
In [92]:
```
data1 = np.random.randn(1000)

plt.hist(data1);
```



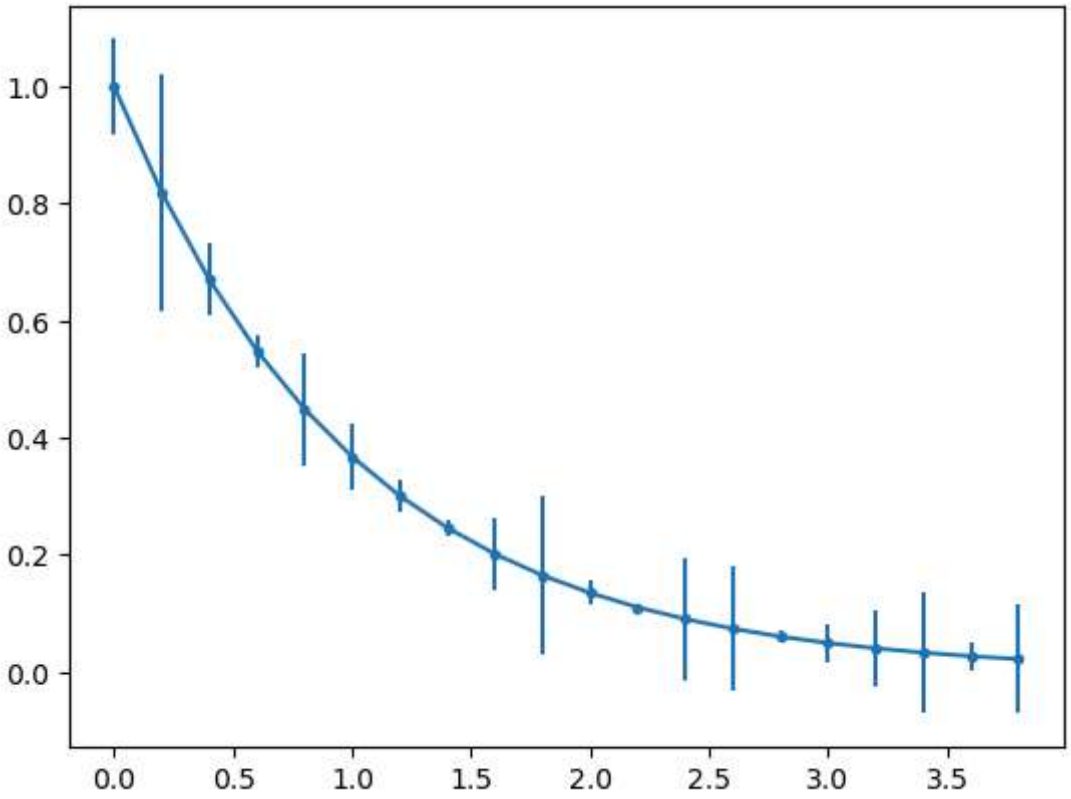
In [95]:
```
data2 = [5. , 25. , 50. , 20. ]
plt.bar(range(len(data2)), data2)
plt.show()
```



In [97]:
```
data2 = [5. , 25. , 50. , 20. ]
plt.barh(range(len(data2)), data2)
plt.show()
```

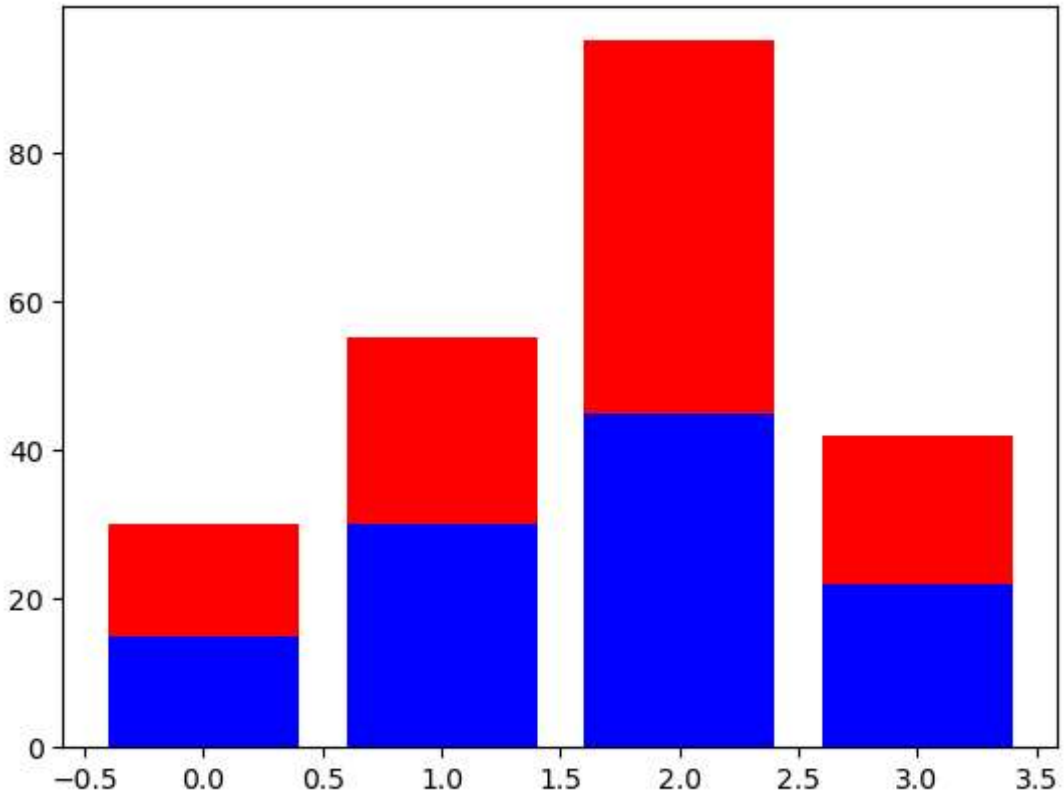


In [99]:
```
x9 = np.arange(0, 4, 0.2)
y9 = np.exp(-x9)

e1 = 0.1 * np.abs(np.random.randn(len(y9)))

plt.errorbar(x9,y9,yerr = e1, fmt = '.-')

plt.show()
```



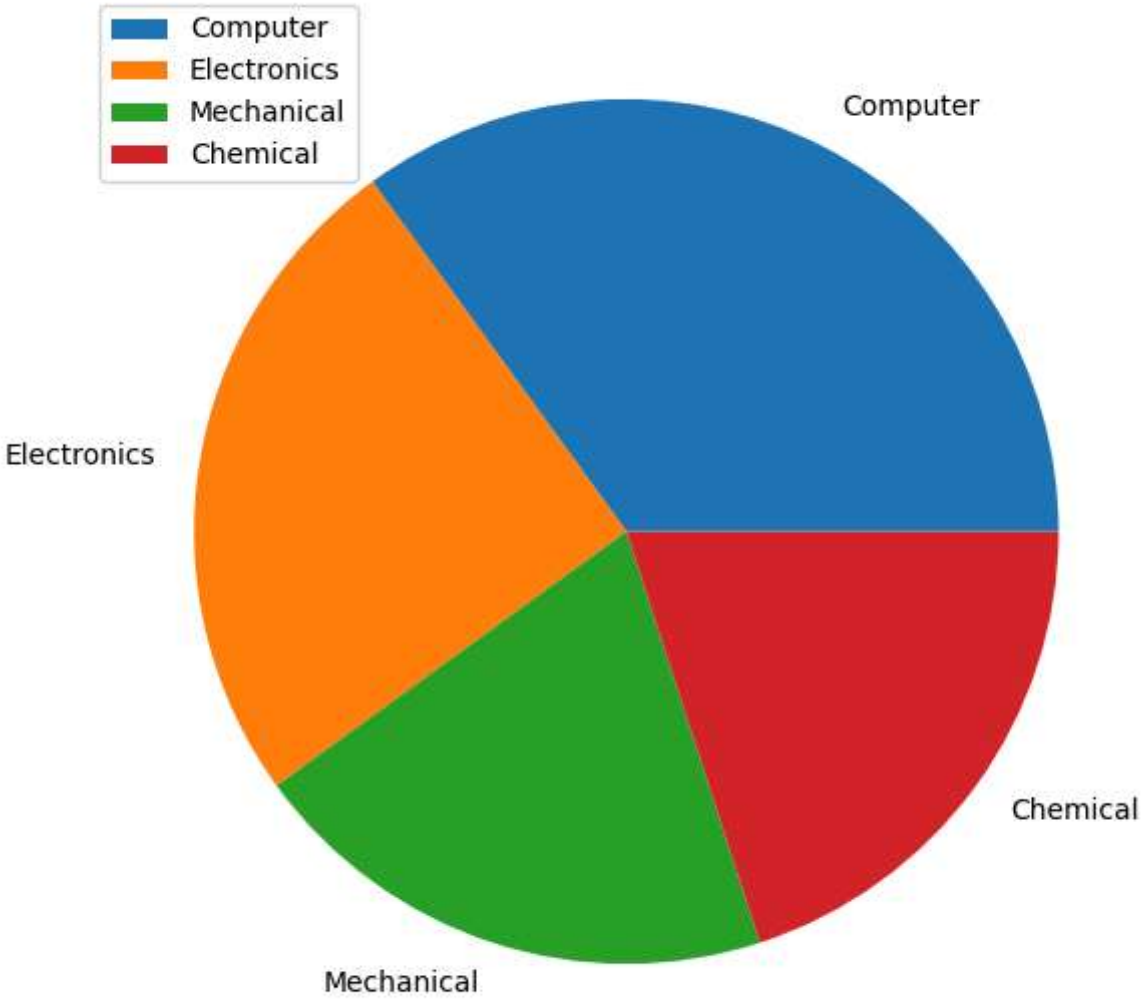
In [101…

```
A = [15., 30., 45., 22.]
B = [15., 25., 50., 20.]

z2 = range(4)

plt.bar(z2, A, color = 'b')
plt.bar(z2, B, color = 'r', bottom = A)

plt.show()
```



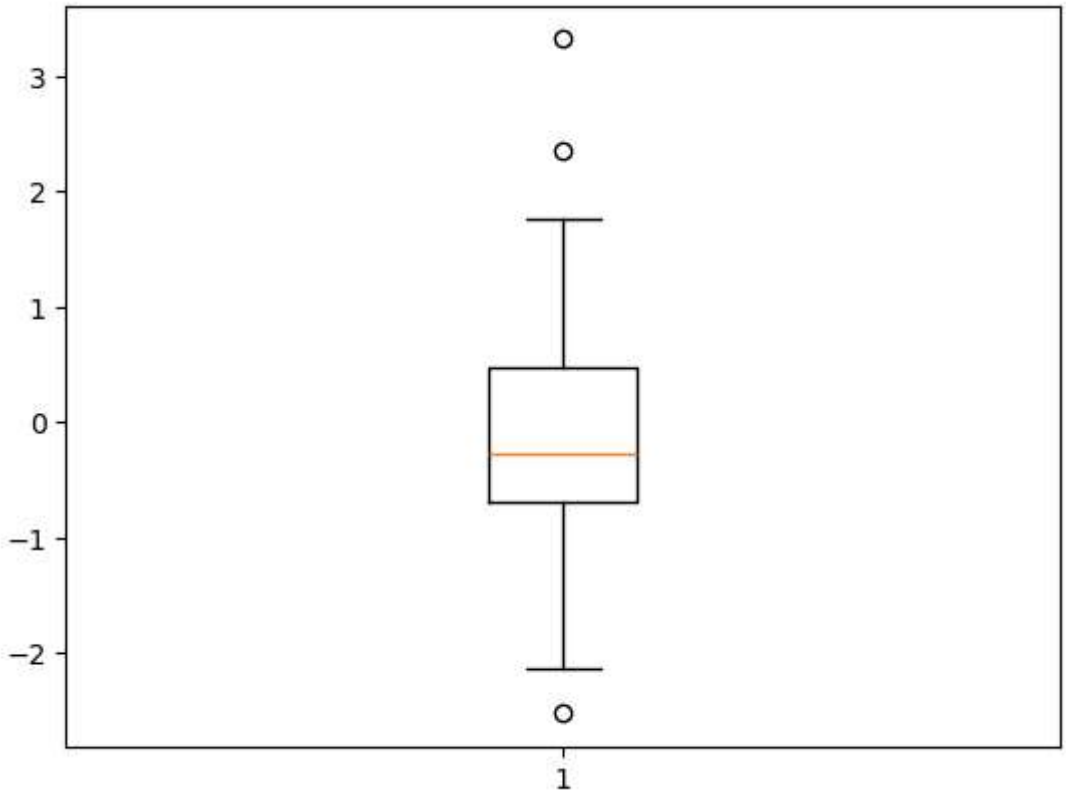
In [109…

```
plt.figure(figsize=(7,7))

x10 = (35, 25 , 20, 20)

labels = ['Computer', 'Electronics', 'Mechanical', 'Chemical']

plt.pie(x10, labels = labels)
plt.legend()
plt.show()
```



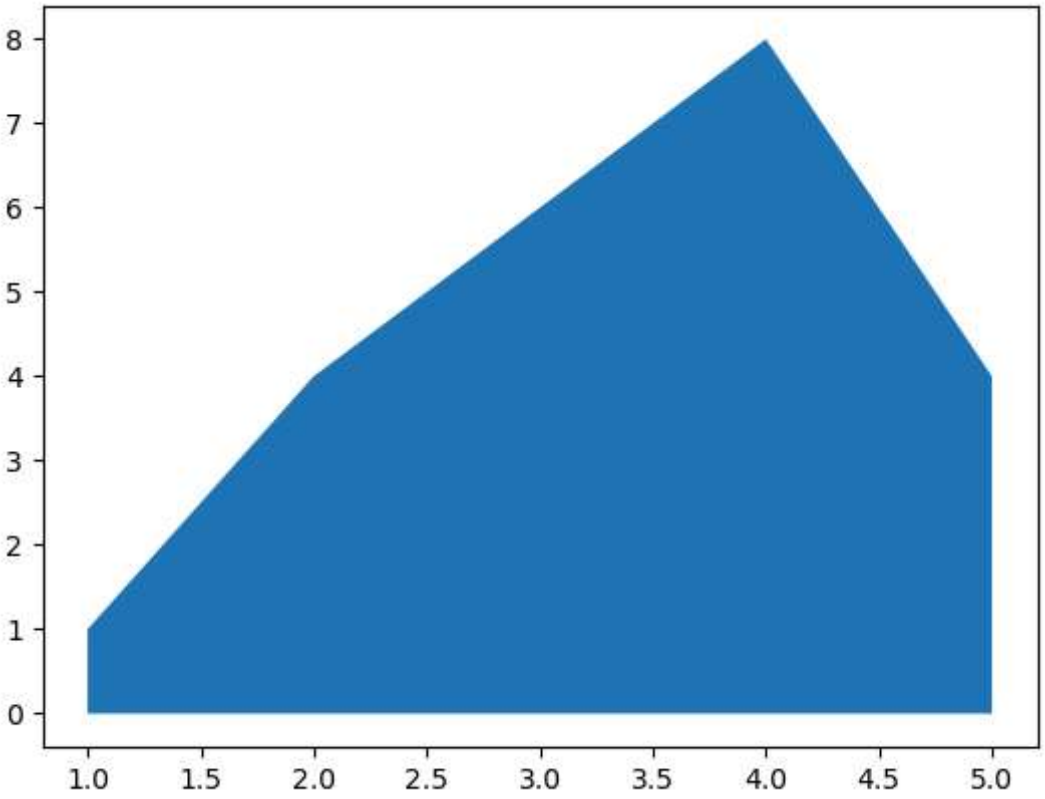
In [111…

```
data3 = np.random.randn(100)
plt.boxplot(data3)
plt.show()
```



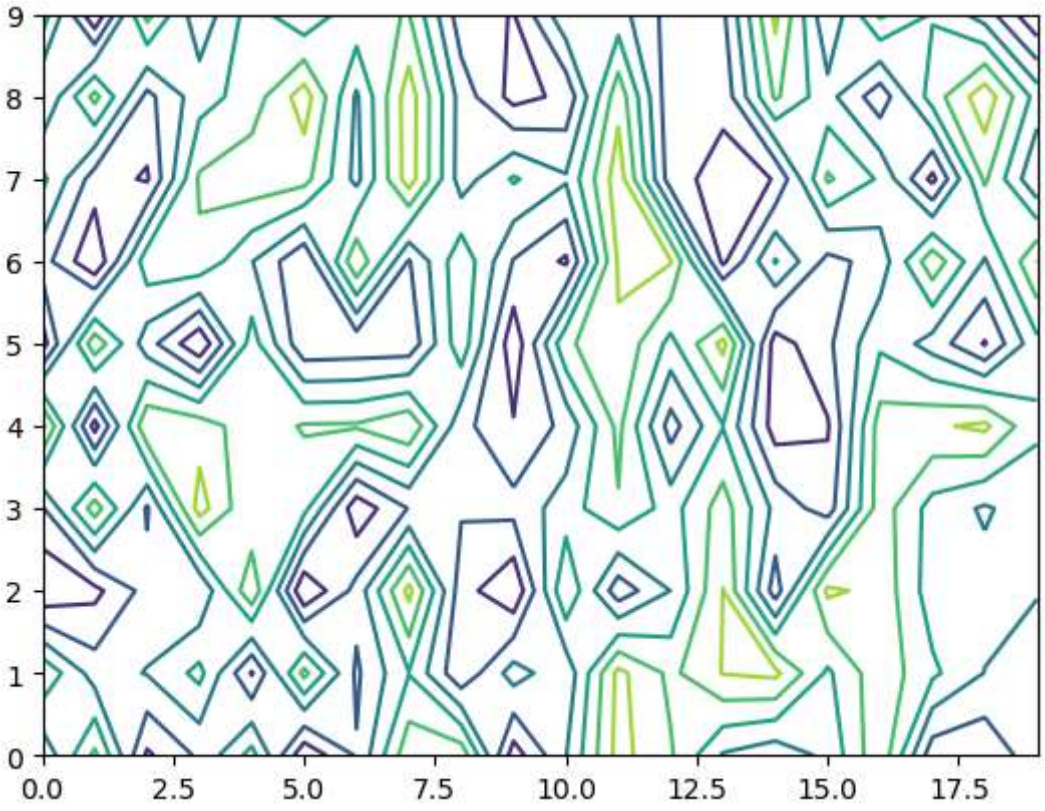
In [113…

```
# create some data

x12 = range(1, 6)
y12 = [1, 4, 6, 8, 4]

# Area Plot
plt.fill_between(x12, y12)
plt.show()
```



In [115…

```
# create a matrix
matrix1 = np.random.rand(10, 20)

cp = plt.contour(matrix1)
plt.show()
```

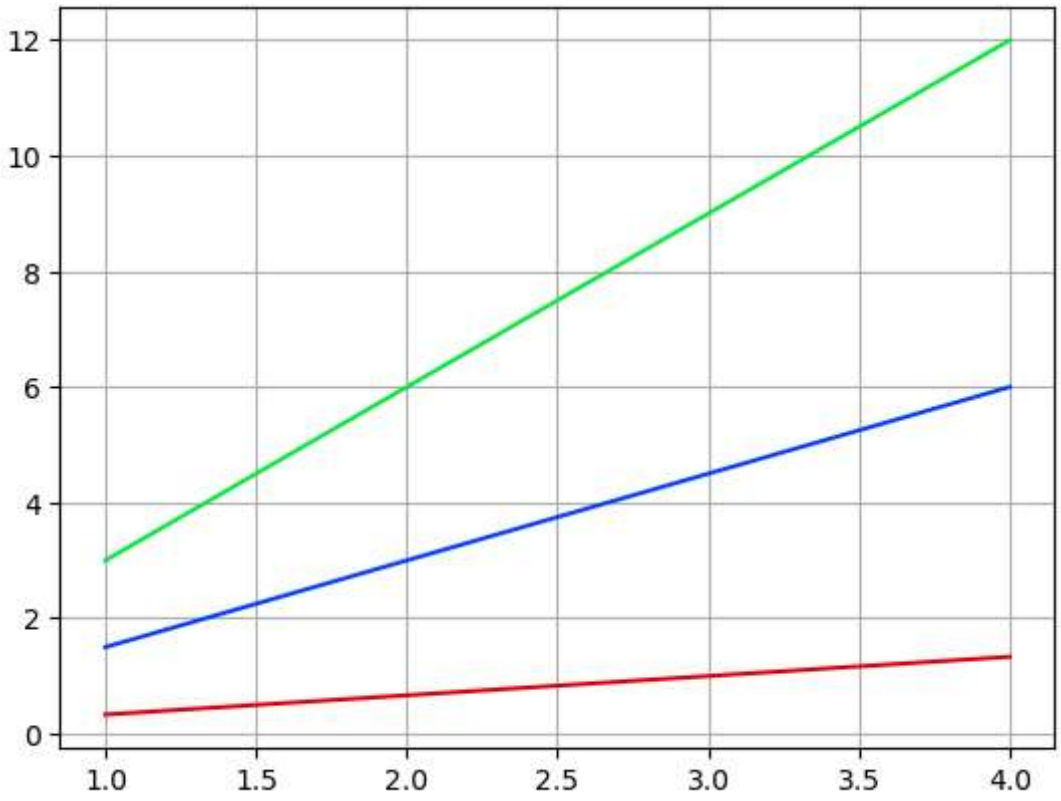


In [123…
```
# view list of all available style

print(plt.style.available)
```

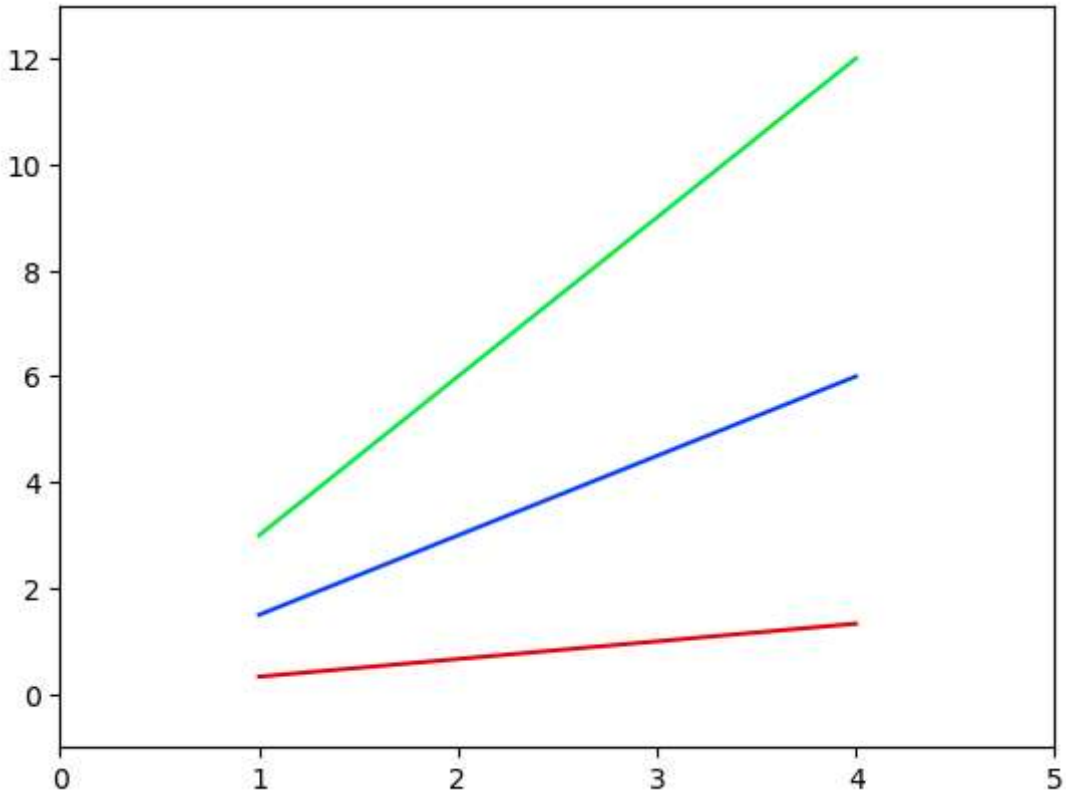
```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

In [125…
```
# Set style for plots

plt.style.use('seaborn-v0_8-bright')
```

In [131…
```
x15 = np.arange(1,5)

plt.plot(x15, x15*1.5, x15, x15*3.0, x15, x15/3.0)
plt.grid(True)
plt.show()
```

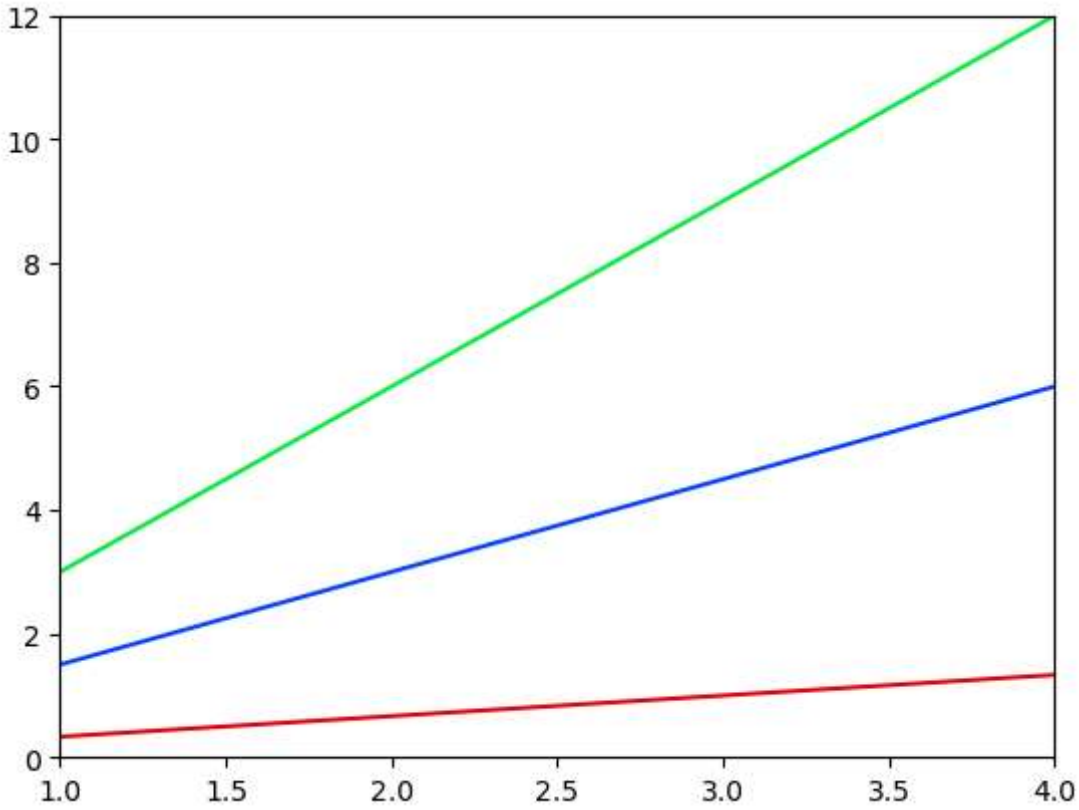


In [135…
```
x15 = np.arange(1,5)

plt.plot(x15, x15*1.5, x15, x15*3.0, x15, x15/3.0)        # we can plot multiple li
plt.axis()      # shows the current axis limits values
plt.axis([0, 5, -1, 13])
plt.show()
```



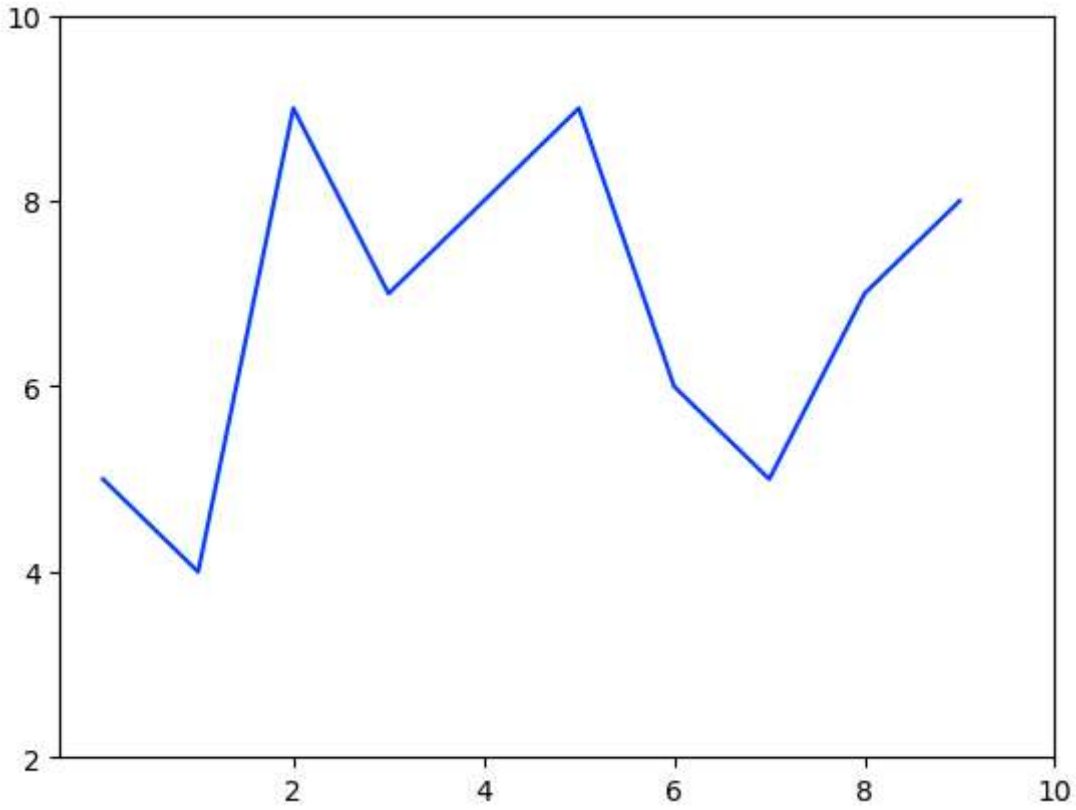
In [139…

```
x15 = np.arange(1,5)

plt.plot(x15, x15*1.5, x15, x15*3.0, x15, x15/3.0)

plt.xlim([1.0, 4.0])
plt.ylim([0.0, 12.0])

plt.show()
```

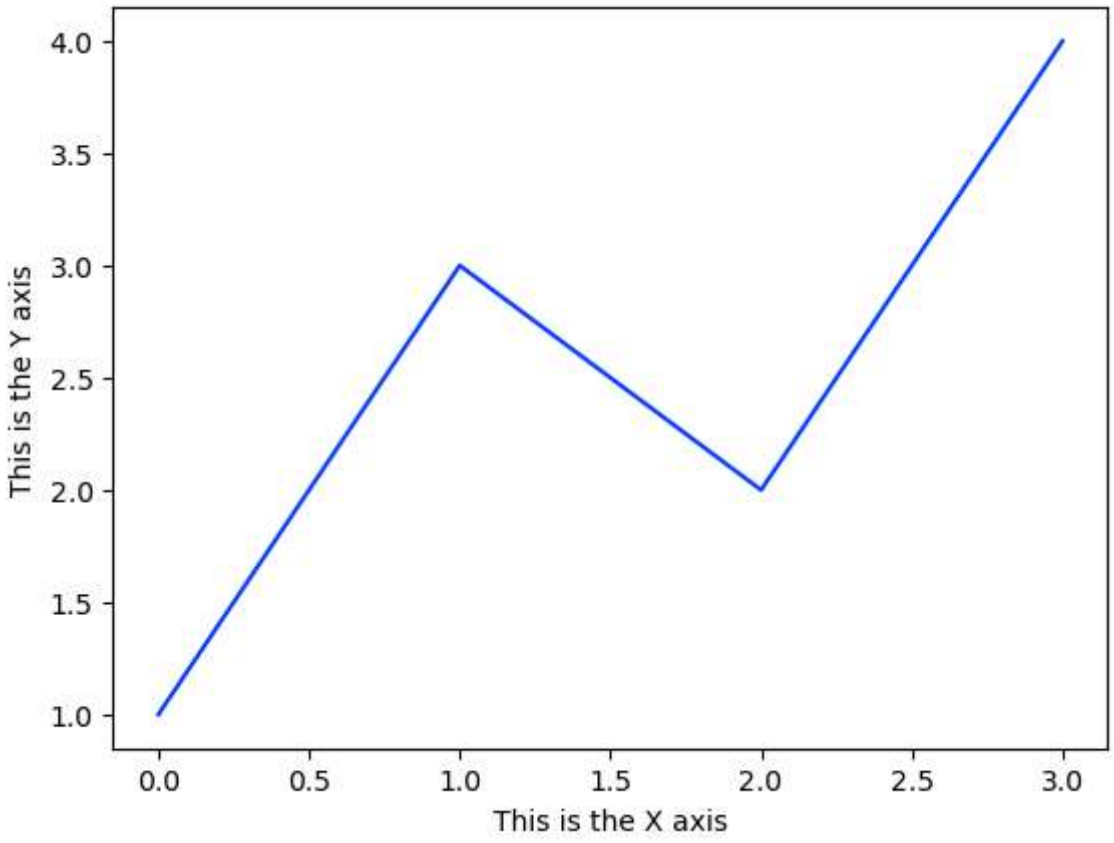


In [143…

```
u = [5, 4, 9, 7, 8, 9, 6, 5, 7, 8]

plt.plot(u)

plt.xticks([2, 4, 6, 8, 10])
plt.yticks([2, 4, 6, 8, 10])
plt.show()
```

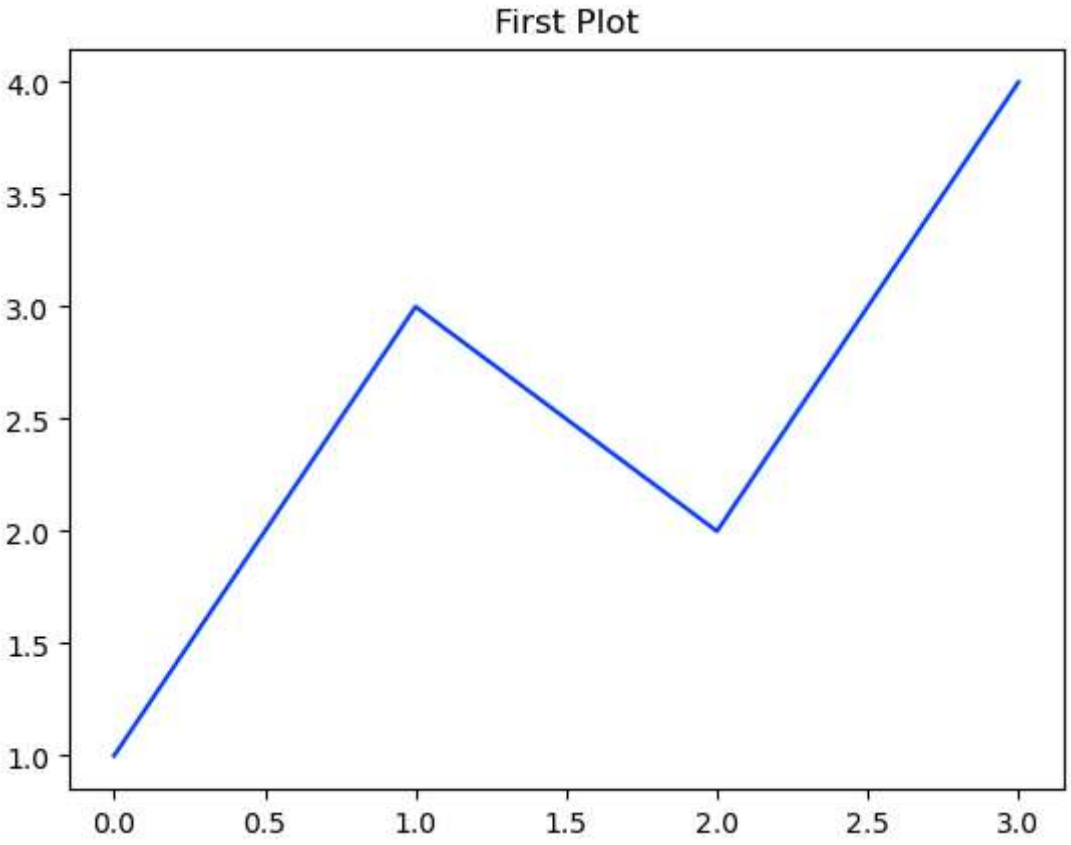


In [145…
```
plt.plot([1, 3, 2, 4])
plt.xlabel('This is the X axis')
plt.ylabel('This is the Y axis')
plt.show()
```



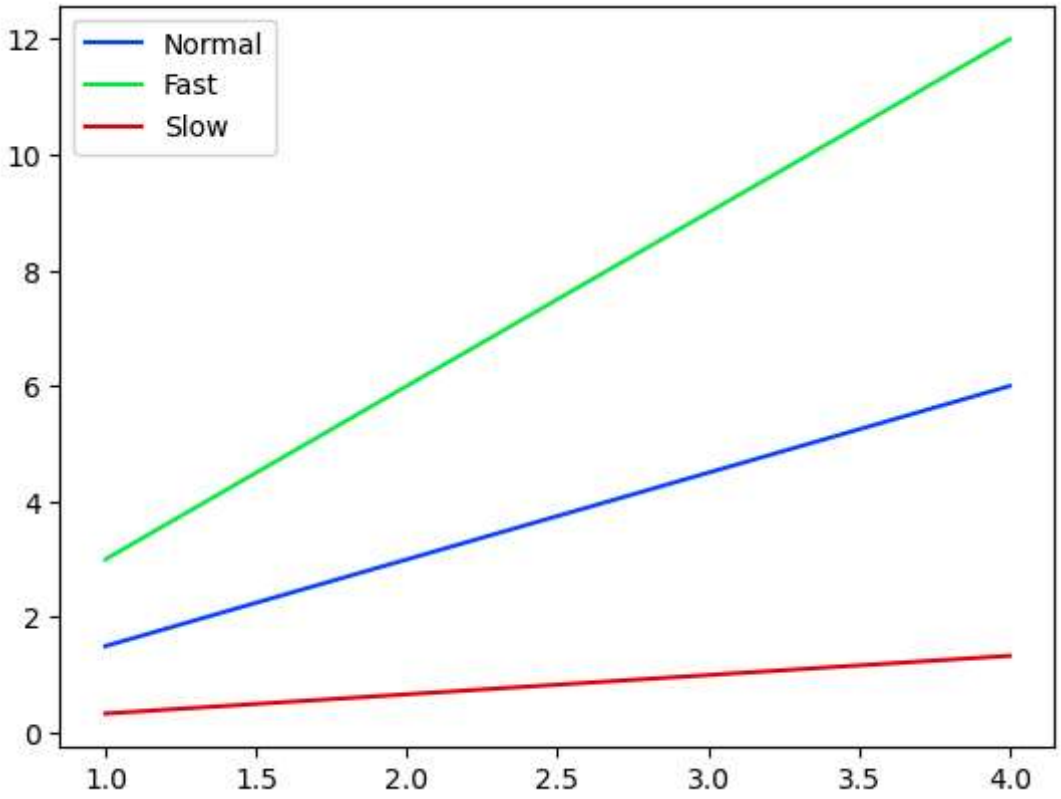
In [151…

```
plt.plot([1,3,2,4])
plt.title('First Plot')
plt.show()
```



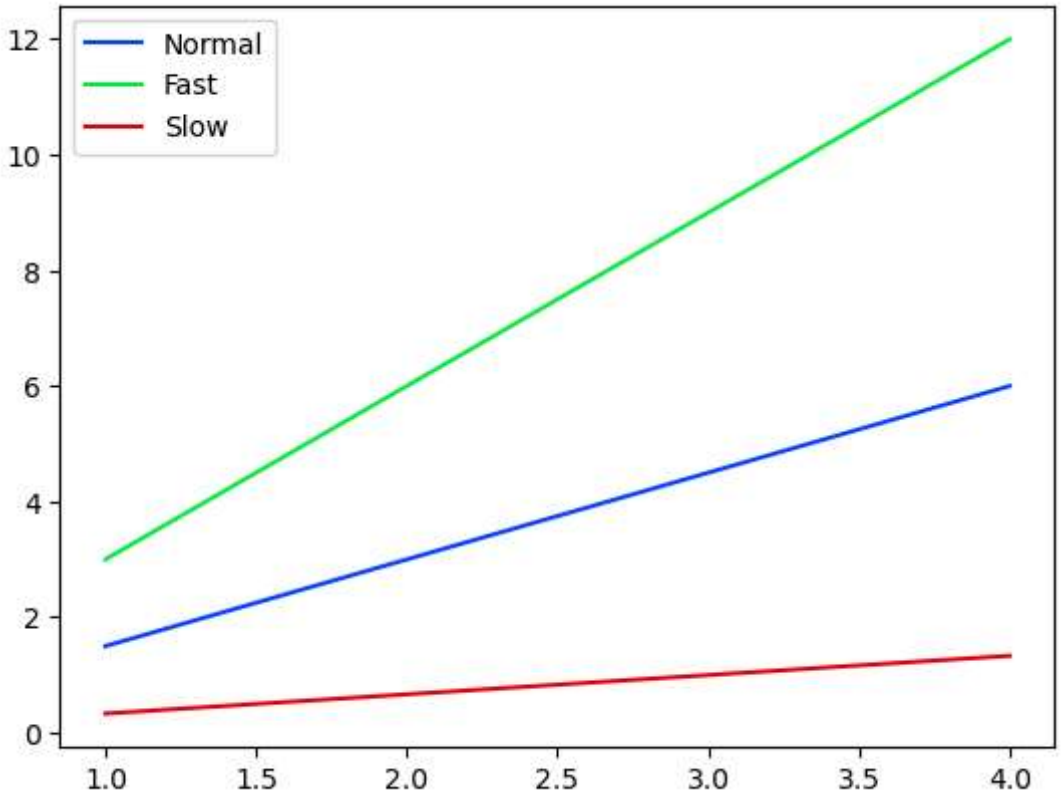
In [157…

```
x15 = np.arange(1,5)

fig,ax = plt.subplots()

ax.plot(x15, x15*1.5)
ax.plot(x15, x15*3.0)
ax.plot(x15, x15/3.0)

ax.legend(['Normal','Fast','Slow']);
```



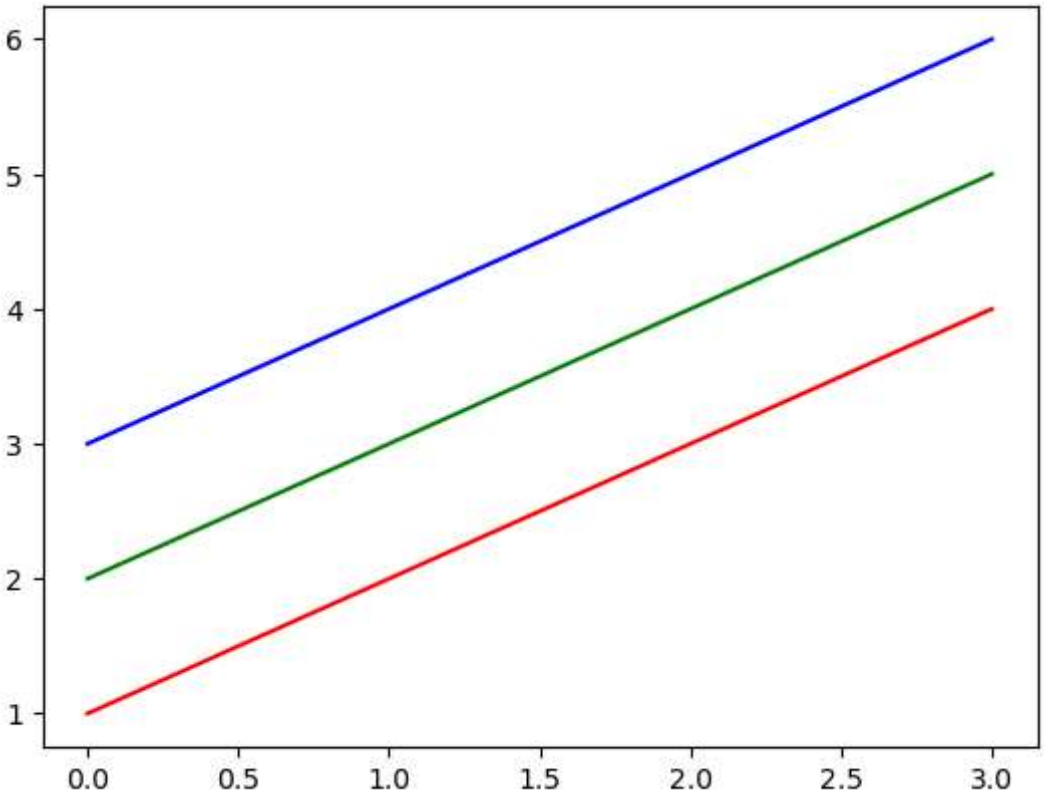
In [159…

```
x15 = np.arange(1,5)

fig,ax = plt.subplots()

ax.plot(x15, x15*1.5, label='Normal')
ax.plot(x15, x15*3.0, label='Fast')
ax.plot(x15, x15/3.0, label='Slow')

ax.legend();
```



In [161…

```
x16 = np.arange(1,5)

plt.plot(x16,'r')
plt.plot(x16+1,'g')
plt.plot(x16+2,'b')

plt.show()
```



In [165…

```
x16 = np.arange(1,5)

plt.plot(x16,'r--',x16+1,'g-.',x16+2,'b:')

plt.show()
```